

Assignment: Builder Pattern

You'll implement **two builders**: Itinerary and HttpRequest. The goal is an immutable product object created.

Part A- Itinerary Builder

Level 1 – Core Implementation

Product fields

- Required
 - travelerName – non-empty string
 - startDate – date
 - endDate – date; must be the same as or after startDate
 - origin – 3-letter IATA (e.g. LHR)
 - destination – 3-letter IATA (e.g. CDG)
- Optional (with defaults)
 - segments – list of Segment (default: empty list)
 - tags – list/set labels (default: empty)
 - budget – money value (amount + ISO currency), optional
 - travelInsurance – Boolean (default: false)

Immutability requirements

- No public setters on the product.
- Expose read-only collections or defensively copy collections on build.
- Primitive/values are final/frozen depending on language.

Fluent builder

- Chaining must read cleanly:

```
Itinerary.builder()
    .travelerName("Alice")
    .origin("LHR").destination("CDG")
    .startDate(2025-10-01).endDate(2025-10-05)
    .build()
```

- If a required field is missing or invalid, build() must fail with a clear error message.

Segment value object (required for use within Itinerary)

- Fields: from, to, departAt, arriveAt, carrier (optional)
- Validation (per segment):
 - from and to are 3-letter codes
 - arriveAt must be **after or equal to** departAt (choose a policy; documenting “must be after” is recommended)

Error message quality

- Include the failing field and a short reason, e.g.,
 - “endDate 2025-10-01 is before startDate 2025-10-05”
 - “origin must be a 3-letter IATA code: ‘London’ given”

Level 2 – Validation & Convenience

Cross-segment / itinerary rules If segments are provided:

- A. **Chronological order:** departAt must be non-decreasing across the list.
- B. **Contiguity:** each segment must connect to the next:
 - segments[i].to == segments[i+1].from
 - segments[i].arriveAt <= segments[i+1].departAt
- C. **Endpoint consistency:**
 - origin == segments[0].from
 - destination == segments[last].to

Convenience methods

- addSegment(segment) – appends one segment (or multiple via repeated calls).
- addTag(tag) – add single tag.
- weekendGetaway(origin, destination) – returns a builder preloaded with:
 - startDate = next Saturday (or documented rule)
 - endDate = startDate + 2 days
 - tags contains “weekend”
 - origin, destination set as provided

Defensive behaviour

- If the builder accepts external lists (e.g. segments([.]), copy on input and copy again on build() to prevent external mutations from affecting the built object.

Typical failure cases to handle

- Gaps/mismatched location between segments: “ Segments must be contiguous: ‘CDG’ -> ‘AMS’ followed by ‘LHR’ -> ‘MAD’”
- Disorder in times: “Segments not in chronological order at index 2 (2025-10-02T08:00 before 2025-10-01T09:00)”

Part B – HttpRequest Builder

Level 1 – Core Implementation

Purpose: Create a **validated, immutable** HTTP request object. **No actual network I/O required.**

Product fields

- **Required**
 - o method- one of GET | POST | PUT | DELETE | PATCH (case policy documented)
 - o url – valid absolute URL
- **Optional (with defaults)**
 - o headers – map/dictionary (default: empty)
 - o body – optional binary/string
 - o timeout – duration (default : 10 seconds)

Immutability

- No setters; headers map exposed read-only or copied.
- Body should be copied (if mutable in your language).

Fluent builder usage example

HttpRequest.builder()

```
.method("POST")
.url("https://api.example.com/items")
.header("Content-Type","application/json")
.body("{\"name\":\"Pen\"}")
.timeout(5 seconds)
.build()
```

Clear errors

- “url is not a valid absolut URL: ‘/relative/path’ given”
- “method must be one of GET, POST , PUT , DELETE, PATCH: ‘FETCH’ given”

Level 2 – Validation & Convenience

Rules

- **GET must not have a body** If body is set while method == GET, build() must fail with a clear message:
 - o “GET requests must not include a body”

Convenience methods

- get(url) – returns a builder preset with method=GET and url=url.
- post(url) – returns a builder preset with method=POST and url=url.
- header(k,v) – adds/replaces a header entry.
- Optional policy: document whether headers are case-insensitive and how you normalize (e.g., canonical casing on build).

Non-requirements (explicit)

- **Do not** implement HTTP transport, retries, redirects, TLS, or streaming.
- **Do not** preform DNS resolution or socket operations.
- Your product object should be serializable/inspectable so a separate networking layer could send it later.

Validation Matrix (quick reference)

Area	Rule	Example error text
Dates	endDate >= startDate	endDate 2025-08-01 is before startDate 2025-08-05
IATA	3 Uppercase letters	Origin must be 3-letter IATA code: ‘Lon’
Segment timing	arriveAt > departAt	Segemtn[1] arriveAt must be after departAT
Segment Order	Non-decreasing by departAt	Segments not chronological at index 2
Contiguity	prev.to == next.from	segment[0].to ‘CDG’ != segment[1].from ‘AMS’

Endpoints	Origin/destination match first/last segment	First segment from 'LGW' != origin 'LHR'
HTTP method	Must be in allowed set	Unsupported method 'Fetch'
GET body	Body forbidden on GET	GET requests must not include a body
URL validity	Absolute URL required	url must be absolute 'api.local/items'

Example scenarios (what students should be able to do)

A. Minimal itinerary

1. Required fields only; resulting object has empty segments, empty tags, travelInsurance=false, budget unset.

B. Multi-segment, contiguous itinerary

1. LHR->AMS, AMS->CDG with matching times and endpoints; passes all validations.

C. Weekend preset

1. HttpRequest.get("https://example.com") has method GET, default timeout 10s, no body: adding body later causes build() to fail.

Glossary & Clarifications

What is a segment (for itinerary)?

A Segment represents on continuous leg of travel between two locations, with a start time and an end time. Think : 'London -> Paris on 2025-10-01 09:00-11:20 via BA' is on segment. A full trip may have multiple segments back to back.

Each segment has:

- from
- to
- departAt
- arriveAt

- **carrier**